

Query Processing – Selection Operation

In query processing, the **file scan** is the lowest-level operator used to access data. File scans are search algorithms that locate and retrieve records that satisfy a given selection condition. In relational systems, a file scan allows an entire relation to be read when the relation is stored in a single dedicated file.

Selections Using File Scans and Indices

The system can perform selection in several ways, depending on available indices and the type of condition.

A1. Linear Search (File Scan)

Algorithm: Scan each file block and test all records to check whether they satisfy the selection condition.

Cost Estimate:

$$\text{Cost} = b_r \text{ block transfers} + 1 \text{ seek}$$

where b_r = number of blocks containing records from relation r .

If selection is on a **key attribute**, scanning can stop once the matching record is found:

$$\text{Cost} = \frac{b_r}{2} \text{ block transfers} + 1 \text{ seek}$$

Notes:

- Can be applied regardless of selection condition or ordering.
- Works even without any index.
- Binary search generally **not suitable** because:
 - Data is not stored consecutively unless indexed.
 - Binary search requires multiple seeks.

Example:

- Query: `SELECT * FROM instructor WHERE salary < 70000;`
- System scans all blocks of `instructor` table.

Remarks: Slowest method, but universally applicable.

A2. Clustering Index on Key (Equality Search)

Algorithm: Retrieve a single record that satisfies the equality condition on a key attribute using a clustering (primary) index.

Cost:

$$\text{Cost} = (h_i + 1) \times (t_S + t_T)$$

where h_i is the height of the B⁺-tree, t_S is the disk seek time, and t_T is the transfer time. The $(h_i + 1)$ term reflects h_i I/O operations to traverse index levels plus one more I/O to fetch the actual data page.

Example:

- Query: `SELECT * FROM student WHERE student_id = 'S123';`
- B⁺-tree index on `student_id` is traversed directly to find the record.

Remarks: Highly efficient for unique key lookups; index height is typically small ($h_i \approx 1$ if cached).

A3. Clustering Index on Non-Key (Equality on Non-Key)

Algorithm: Retrieve multiple records using a clustering index when the condition is an equality on a non-key attribute.

Records are stored consecutively because the file is sorted on the clustering key.

Cost:

$$\text{Cost} = h_i \times (t_S + t_T) + t_S + b \times t_T$$

where:

- h_i = index height
- b = number of blocks containing matching records

Explanation of Cost Formula: The term $h_i(t_S + t_T)$ is the cost to traverse the B⁺-tree index down to the first matching leaf entry. The extra t_S represents one seek to position the disk head at the start of the contiguous block range. Finally, $b \times t_T$ is the cost to sequentially read the b consecutive data blocks containing all matching records.

Example:

- Query: `SELECT * FROM instructor WHERE dept_name = 'CSE';`
- All CSE instructor records are on consecutive blocks.

Remarks: More efficient than linear scan since the matching records are contiguous.

A4. Secondary Index (Equality on Key or Non-Key)

Algorithm: Use a secondary (non-clustering) index to find records satisfying equality conditions on the search-key.

Case 1 – Equality on Key:

$$\text{Cost} = (h_i + 1) \times (t_S + t_T)$$

Case 2 – Equality on Non-Key: Each of the n matching records may be on a different block:

$$\text{Cost} = (h_i + n) \times (t_S + t_T)$$

Example:

- Query: `SELECT * FROM instructor WHERE city = 'Dhaka';`
- Secondary index on `city` locates all matching records, possibly scattered across blocks.

Remarks:

- Can become expensive if many records match (each may need a separate I/O).
- Often mitigated by buffer caching.

Concept: Access Paths

Index structures are called **access paths**, as they define how data can be located efficiently.

- **Clustering Index:** Records are stored in physical order of index key.
- **Secondary Index:** Records are not stored in index key order.

Summary of Selection Methods

Method	Type	When Applicable
A1	Linear Search	No index available
A2	Clustering Index on Key	Equality on key
A3	Clustering Index on Non-Key	Equality on non-key
A4	Secondary Index	Equality on any indexed field

Typical Access Costs (4 KB Blocks)

- High-end Magnetic Disk: $t_S = 4 \text{ ms}$, $t_T = 0.1 \text{ ms}$
- SSD: $t_S = 20\text{--}90 \mu\text{s}$, $t_T = 2\text{--}10 \mu\text{s}$

Note: In practice, index internal nodes are cached, so effective $h_i \approx 1$.